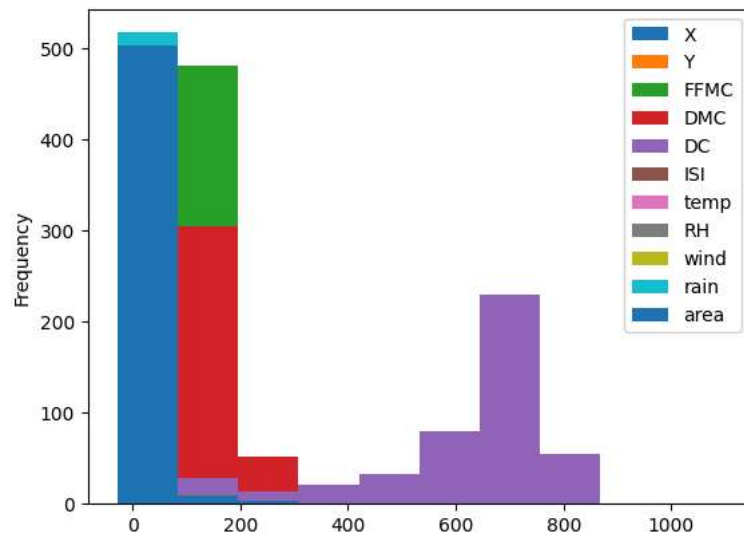


```
import pandas as pd
import numpy as np
```

```
df=pd.read_excel('forestfires.xlsx',sheet_name='forestfires')
```

```
import matplotlib.pyplot as plt
df.plot.hist()
```

<Axes: ylabel='Frequency'>



```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 517 entries, 0 to 516
Data columns (total 13 columns):
#   Column  Non-Null Count  Dtype
---  ------  -
0    X        517 non-null      int64
1    Y        517 non-null      int64
2   month    514 non-null      object
3   day      517 non-null      object
4   FFMC     517 non-null      float64
5   DMC      517 non-null      float64
6   DC       517 non-null      float64
7   ISI      517 non-null      float64
8   temp     517 non-null      float64
9   RH       515 non-null      float64
10  wind     517 non-null      float64
11  rain     517 non-null      float64
12  area     517 non-null      float64
dtypes: float64(9), int64(2), object(2)
memory usage: 52.6+ KB
None
```

```
print(df.describe())
```

	X	Y	FFMC	DMC	DC	ISI \
count	517.000000	517.000000	517.000000	517.000000	517.000000	517.000000
mean	4.669246	4.299807	90.644681	110.872340	547.940039	9.021663
std	2.313778	1.229900	5.520111	64.046482	248.066192	4.559477
min	1.000000	2.000000	18.700000	1.100000	7.900000	0.000000
25%	3.000000	4.000000	90.200000	68.600000	437.700000	6.500000
50%	4.000000	4.000000	91.600000	108.300000	664.200000	8.400000
75%	7.000000	5.000000	92.900000	142.400000	713.900000	10.800000
max	9.000000	9.000000	96.200000	291.300000	860.600000	56.100000

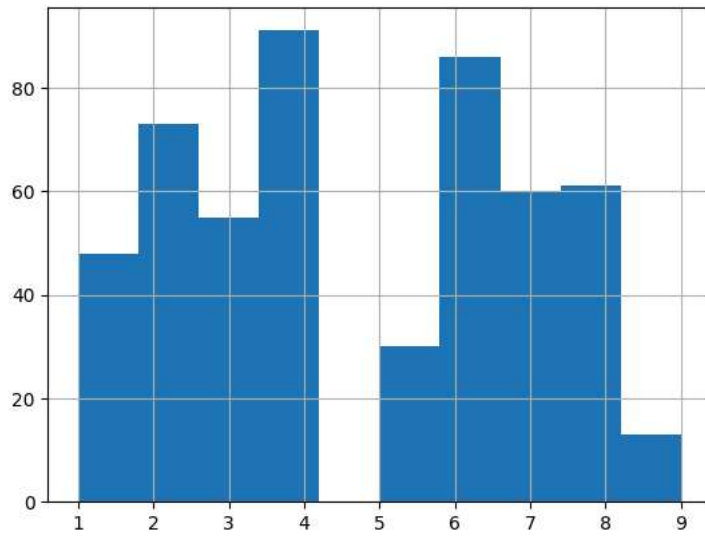
	temp	RH	wind	rain	area
count	517.000000	515.000000	517.000000	517.000000	517.000000
mean	18.889168	44.153398	4.017602	0.021663	12.750155
std	5.806625	16.572144	1.791653	0.295959	63.675384
min	2.200000	-27.000000	0.400000	0.000000	-14.290000
25%	15.500000	33.000000	2.700000	0.000000	0.000000
50%	19.300000	42.000000	4.000000	0.000000	0.470000
75%	22.800000	53.000000	4.900000	0.000000	6.440000
max	33.300000	100.000000	9.400000	6.400000	1090.840000

```
df.columns
```

```
Index(['X', 'Y', 'month', 'day', 'FFMC', 'DMC', 'DC', 'ISI', 'temp', 'RH',  
      'wind', 'rain', 'area'],  
      dtype='object')
```

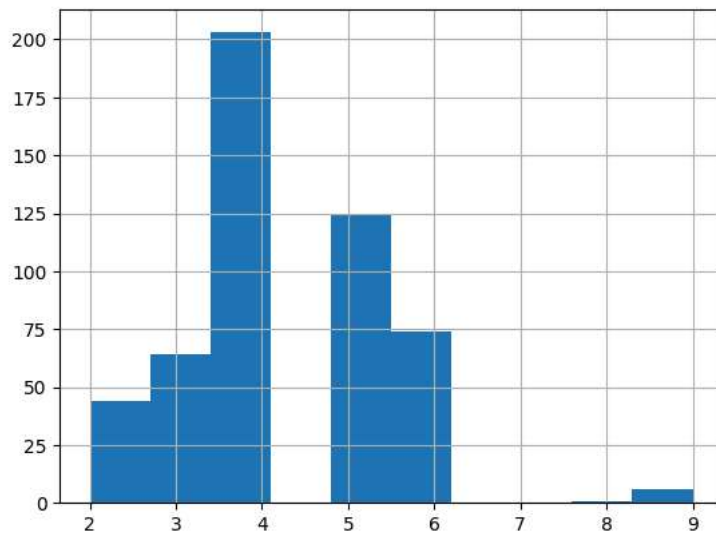
```
import matplotlib.pyplot as plt  
df['X'].hist()
```

<Axes: >



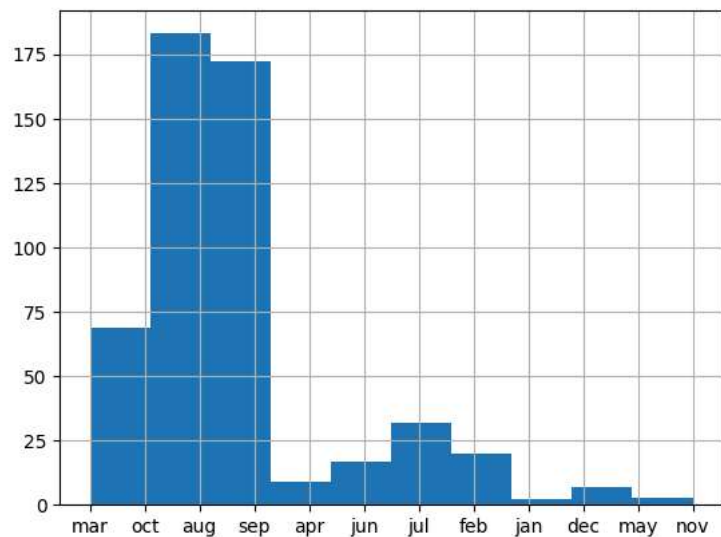
```
df['Y'].hist()
```

<Axes: >



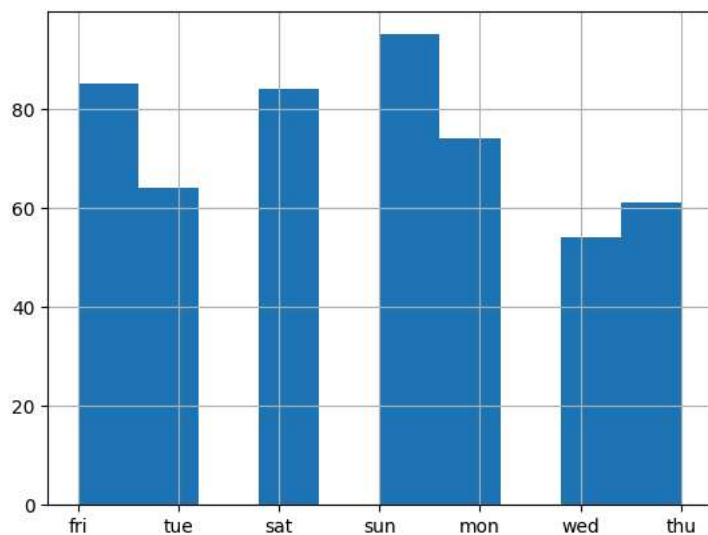
```
df['month'].hist()
```

<Axes: >



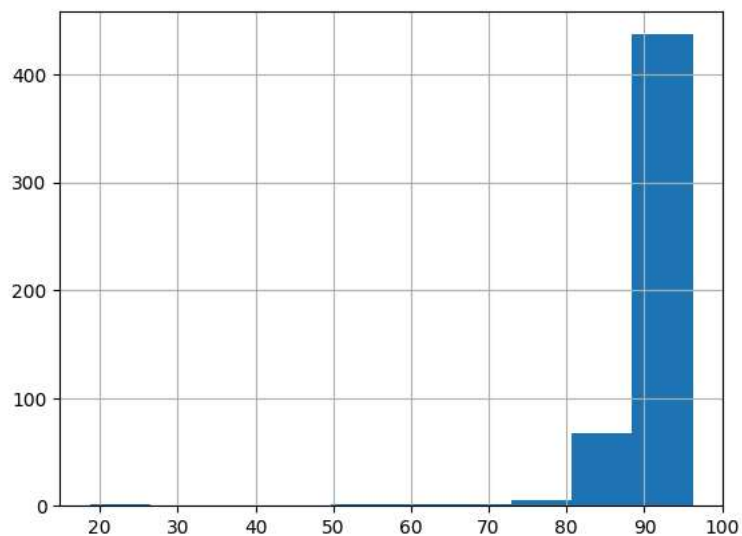
df['day'].hist()

<Axes: >



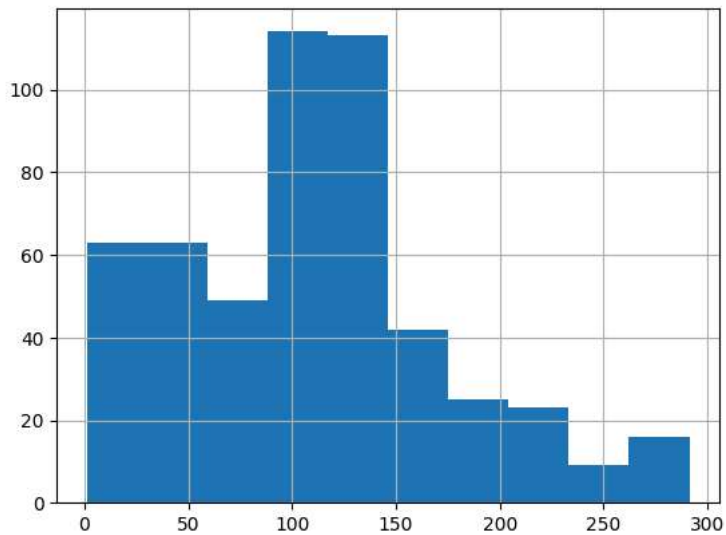
df['FFMC'].hist()

<Axes: >

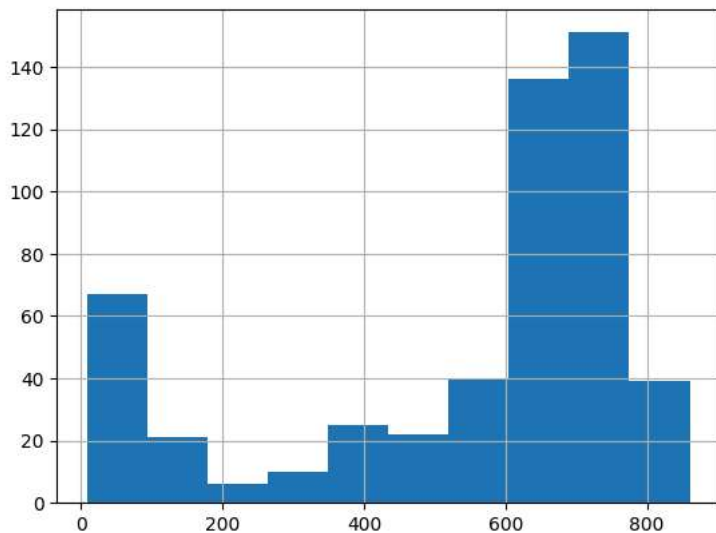


df['DMC'].hist()

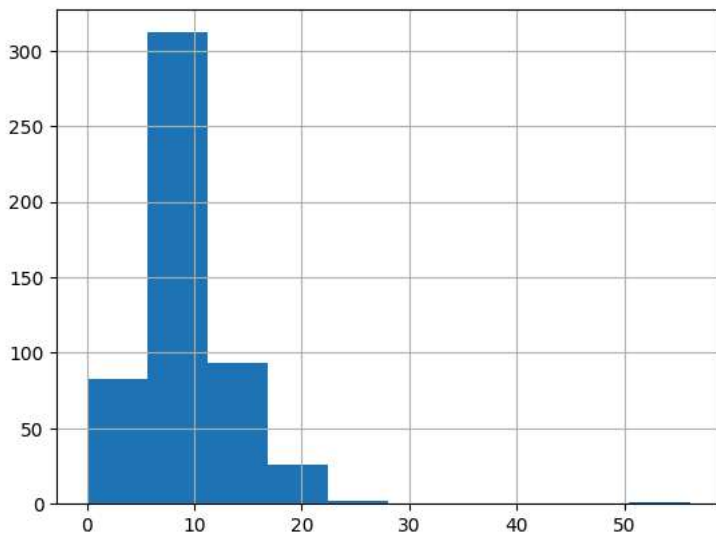
<Axes: >

`df['DC'].hist()`

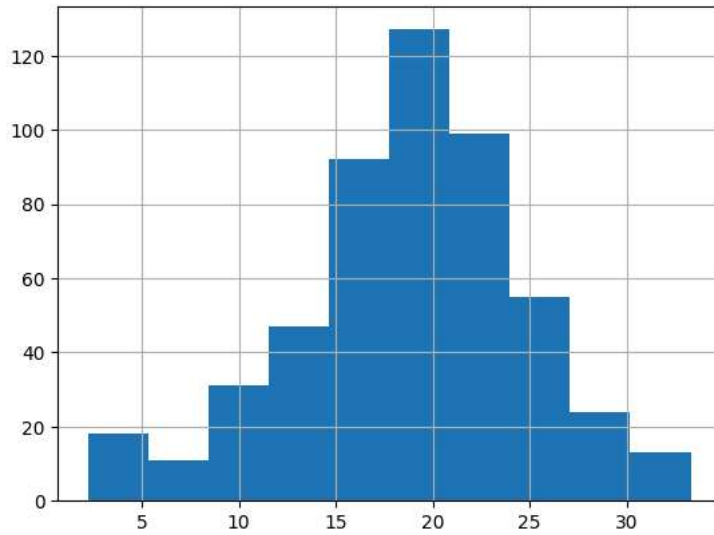
<Axes: >

`df['ISI'].hist()`

<Axes: >

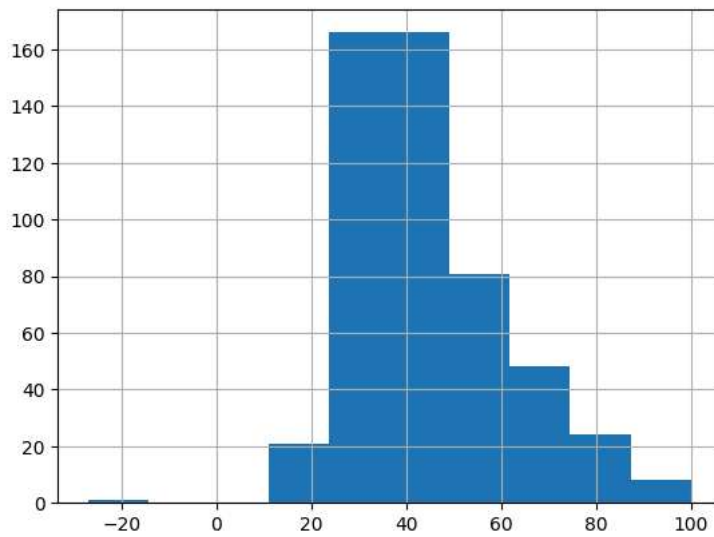
`df['temp'].hist()`

<Axes: >



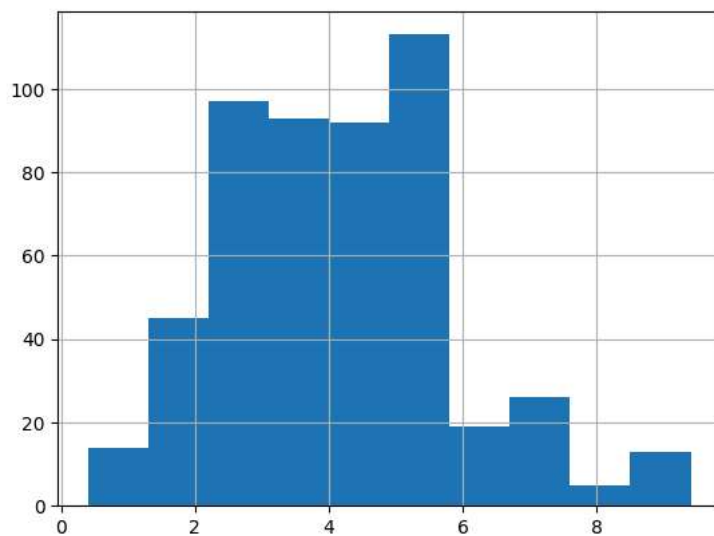
df['RH'].hist()

<Axes: >



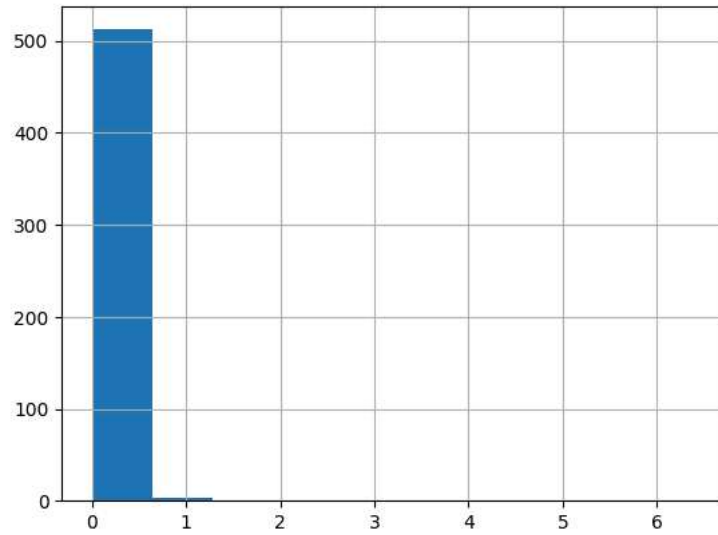
df['wind'].hist()

<Axes: >



df['rain'].hist()

<Axes: >



```
df['rain'].describe()
```

	rain
count	517.000000
mean	0.021663
std	0.295959
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	6.400000

```
dtype: float64
```

```
numCeros=(df['area']==0).sum()  
print(numCeros)
```

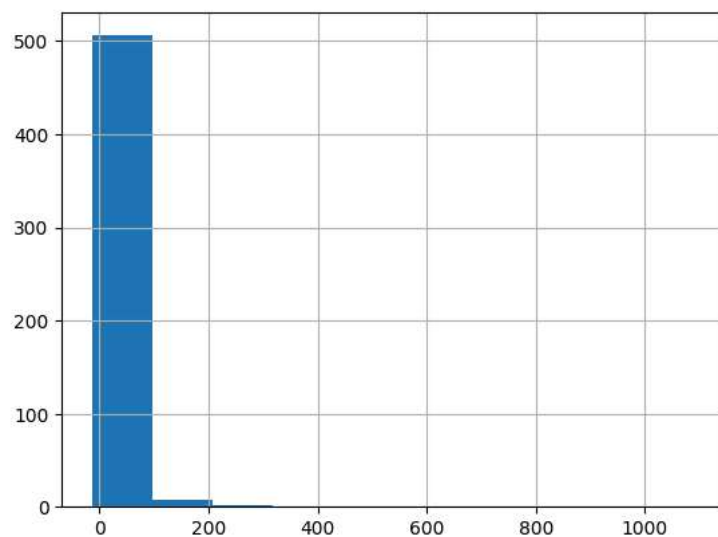
```
247
```

```
num=df['rain'].unique()  
print(num)
```

```
[0.  0.2  1.   6.4  0.8  0.4  1.4]
```

```
df['area'].hist()
```

<Axes: >



```
numCeros=(df['area']==0).sum()
print(numCeros)
```

247

```
num=df['area'].unique()
print(num)
```

```
[ 0.00000e+00  3.60000e-01  4.30000e-01  4.70000e-01  5.50000e-01
 6.10000e-01  7.10000e-01  7.70000e-01  9.00000e-01  9.50000e-01
 9.60000e-01  1.07000e+00  1.12000e+00  1.19000e+00  1.36000e+00
 1.43000e+00  1.46000e+00  1.56000e+00  1.61000e+00  1.63000e+00
 1.64000e+00  1.69000e+00  1.75000e+00  1.90000e+00  1.94000e+00
 1.95000e+00  2.01000e+00  2.14000e+00  2.29000e+00  2.51000e+00
 2.53000e+00  2.55000e+00  2.57000e+00  2.69000e+00  2.74000e+00
 3.07000e+00  3.50000e+00  4.53000e+00  4.61000e+00  4.69000e+00
 4.88000e+00  5.23000e+00  5.33000e+00  5.44000e+00  6.38000e+00
 6.83000e+00  6.96000e+00  7.04000e+00  7.19000e+00  7.30000e+00
 7.40000e+00  8.24000e+00  8.31000e+00  8.68000e+00  8.71000e+00
 9.41000e+00  1.00100e+01  1.00200e+01  1.09300e+01  1.10600e+01
 1.12400e+01  1.13200e+01  1.15300e+01  1.21000e+01  1.30500e+01
 1.37000e+01  1.39900e+01  1.45700e+01  1.54500e+01  1.72000e+01
 1.92300e+01  2.34100e+01  2.42300e+01  2.60000e+01  2.61300e+01
 2.73500e+01  2.86600e+01  2.94800e+01  3.03200e+01  3.17200e+01
 3.18600e+01  3.20700e+01  3.58800e+01  3.68500e+01  3.70200e+01
 3.77100e+01  4.85500e+01  4.93700e+01  5.83000e+01  6.41000e+01
 7.13000e+01  8.84900e+01  9.51800e+01  1.03390e+02  1.05660e+02
 1.54880e+02  1.96480e+02  2.00940e+02  2.12880e+02  1.09084e+03
 1.01300e+01  2.87000e+00  7.60000e-01  9.00000e-02  7.50000e-01
 2.47000e+00  6.80000e-01  2.40000e-01  2.10000e-01  1.52000e+00
 1.03400e+01  8.02000e+00  1.38000e+00  8.85000e+00  3.30000e+00
 4.25000e+00  6.54000e+00  7.90000e-01  1.70000e-01  4.40000e+00
 5.20000e-01  9.27000e+00  3.09000e+00  8.98000e+00  1.11900e+01
 5.38000e+00  1.78500e+01  1.07300e+01  2.20300e+01  9.77000e+00
 2.47700e+01  1.10000e+00  2.42400e+01  8.00000e+00  2.64000e+00
 8.64500e+01  6.57000e+00  3.52000e+00  4.10000e-01  5.18000e+00
 -1.42900e+01  1.58000e+00  3.78000e+00  4.41000e+00  3.43600e+01
 7.21000e+00  1.01000e+00  2.18000e+00  4.42000e+00  3.33000e+00
 6.58000e+00  1.56400e+01  1.12200e+01  2.13000e+00  5.60400e+01
 7.48000e+00  1.47000e+00  3.93000e+00  6.10000e+00  5.83000e+00
 2.81900e+01  3.71000e+00  7.31000e+00  2.03000e+00  1.72000e+00
 5.97000e+00  1.30600e+01  1.26000e+00  8.12000e+00  1.09000e+00
 3.94000e+00  2.93000e+00  5.65000e+00  2.00300e+01  1.26400e+01
 1.83000e+01  3.93500e+01  1.74630e+02  7.73000e+00  1.63300e+01
 5.86000e+00  4.28700e+01  1.21800e+01  1.60000e+01  2.45900e+01
 2.87400e+01  9.96000e+00  3.01800e+01  7.07600e+01  5.17800e+01
 3.64000e+00  3.63000e+00  8.16000e+00  4.95000e+00  6.04000e+00
 3.95000e+00  7.80000e+00  4.62000e+00  7.46280e+02  7.02000e+00
 2.44000e+00  3.05000e+00  1.85760e+02  6.30000e+00  7.20000e-01
 4.96000e+00  2.35000e+00  3.20000e+00  6.36000e+00  1.53400e+01
 5.40000e-01  6.43000e+00  3.30000e-01  1.23000e+00  3.35000e+00
 9.71000e+00  8.27500e+01  3.32000e+00  5.39000e+00  6.84000e+00
 3.18000e+00  5.55000e+00  6.61000e+00  6.11300e+01  3.84800e+01
 7.03200e+01  1.00800e+01  3.19000e+00  1.76000e+00  7.36000e+00
 2.21000e+00  2.78530e+02  2.75000e+00  1.29000e+00  2.64300e+01
 2.07000e+00  2.00000e+00  1.64000e+01  4.67000e+01  4.33200e+01
 8.59000e+00  2.77000e+00  1.46800e+01  4.05400e+01 -1.08200e+01
 4.95900e+01  5.80000e+00  2.17000e+00  6.44000e+00  5.42900e+01
 1.11600e+01]
```

```
percentage_nulos=(df.isnull().sum()/len(df))*100
num_nulos=df.isnull().sum()
```

```
for c in df.columns:
    print(c,"-->",percentage_nulos[c],"%")
```

```
X --> 0.0 %
Y --> 0.0 %
month --> 0.5802707930367506 %
day --> 0.0 %
FFMC --> 0.0 %
DMC --> 0.0 %
DC --> 0.0 %
ISI --> 0.0 %
temp --> 0.0 %
RH --> 0.3868471953578337 %
wind --> 0.0 %
rain --> 0.0 %
area --> 0.0 %
```

```
for c in df.columns:
    print(c,"-->",num_nulos[c])
```

```
X --> 0
Y --> 0
month --> 3
```

```
day --> 0
FFMC --> 0
DMC --> 0
DC --> 0
ISI --> 0
temp --> 0
RH --> 2
wind --> 0
rain --> 0
area --> 0
```

```
mediaRH=df['RH'].mean()
modeMonth=df['month'].mode()[0]
df['RH']=df['RH'].fillna(mediaRH)
df['month']=df['month'].fillna(modeMonth)
```

```
percentage_nulos=(df.isnull().sum()/len(df))*100
num_nulos=df.isnull().sum()
```

```
for c in df.columns:
    print(c,"-->",percentage_nulos[c],"%")
```

```
X --> 0.0 %
Y --> 0.0 %
month --> 0.0 %
day --> 0.0 %
FFMC --> 0.0 %
DMC --> 0.0 %
DC --> 0.0 %
ISI --> 0.0 %
temp --> 0.0 %
RH --> 0.0 %
wind --> 0.0 %
rain --> 0.0 %
area --> 0.0 %
```

```
for col in df.columns:
    numUnico=df[col].nunique()
    if numUnico<3 :
        print("La columna:",col," es semiconstante")

    print("La columna:",col," es normal")
```

```
La columna: X es normal
La columna: Y es normal
La columna: month es normal
La columna: day es normal
La columna: FFMC es normal
La columna: DMC es normal
La columna: DC es normal
La columna: ISI es normal
La columna: temp es normal
La columna: RH es normal
La columna: wind es normal
La columna: rain es normal
La columna: area es normal
```

```
duplicados_totales=df.duplicated().sum()
print("Filas duplicadas", duplicados_totales)
```

```
Filas duplicadas 4
```

```
df=df.drop_duplicates()
```

```
duplicados_totales=df.duplicated().sum()
if(duplicados_totales>0):
    print("Filas duplicadas", duplicados_totales)
else:
    print("Buena buena")
```

```
Buena buena
```



```
variables_no_negativas=['FFMC', 'DMC', 'DC', 'ISI', 'temp', 'RH', 'wind', 'rain', 'area']

for c in variables_no_negativas:
    negativos=(df[c]<0).sum()
    if negativos>0:
        print("Errores en ", c)
        print("#Errores: ", negativos)
```

```
Errores en RH
#Errores: 1
Errores en area
#Errores: 2
```

```
df['RH']=np.where(df['RH']<0,0,df['RH'])
```

```
df['area']=np.where(df['area']<0,0,df['area'])
```

```
variables_no_negativas=['FFMC', 'DMC', 'DC', 'ISI', 'temp', 'RH', 'wind', 'rain', 'area']

for c in variables_no_negativas:
    negativos=(df[c]<0).sum()
    if negativos>0:
        print("Errores en ", c)
        print("#Errores: ", negativos)
```

OUTLIERS

```
numeric_cols=['FFMC', 'DMC', 'DC', 'ISI', 'temp', 'RH', 'wind', 'rain', 'area']
```

```
df.head()
```

	X	Y	month	day	FFMC	DMC	DC	ISI	temp	RH	wind	rain	area
0	7	5	mar	fri	86.2	26.2	94.3	5.1	8.2	51.0	6.7	0.0	0.0
1	7	4	oct	tue	90.6	35.4	669.1	6.7	18.0	33.0	0.9	0.0	0.0
2	7	4	oct	sat	90.6	43.7	686.9	6.7	14.6	33.0	1.3	0.0	0.0
3	8	6	mar	fri	91.7	33.3	77.5	9.0	8.3	97.0	4.0	0.2	0.0
4	8	6	mar	sun	89.3	51.3	102.2	9.6	11.4	99.0	1.8	0.0	0.0

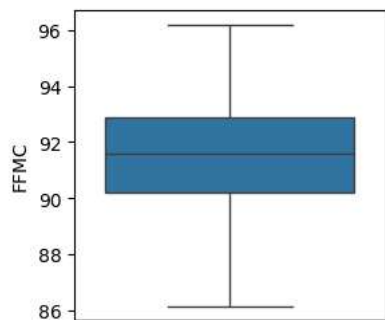
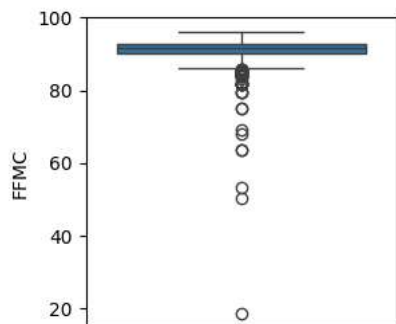
```
def detectaroutliers(df, column):
    Q1=df[column].quantile(0.25)
    Q3=df[column].quantile(0.75)
    IQR=Q3-Q1
    rangoInf=Q1-IQR*1.5
    rangoSup=Q3+IQR*1.5
    outliers=df[(df[column]<rangoInf)|(df[column]>rangoSup)]
    return outliers,rangoInf,rangoSup,Q1,Q3
```

```
import seaborn as sns
```

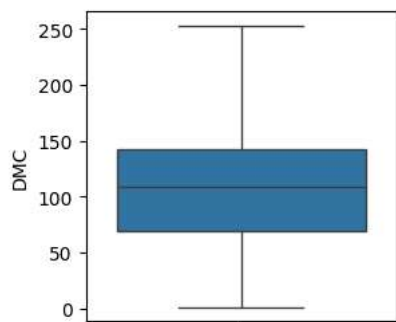
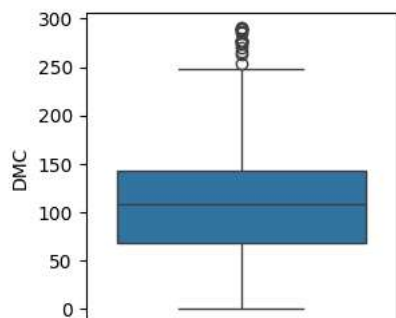
```
for c in numeric_cols:
    outliers,rangoInf,rangoSup,Q1,Q3=detectaroutliers(df,c)
    if(len(outliers)>0):
        print("La columna: ",c," tiene ",len(outliers)," outliers")
        print("Rango inf: ",rangoInf,"Rango sup: ",rangoSup)
        plt.figure(figsize=(3, 3))
        sns.boxplot(y=df[c])
        plt.show()
        df[c]=df[c].clip(lower=rangoInf,upper=rangoSup)

    plt.figure(figsize=(3, 3))
    sns.boxplot(y=df[c])
    plt.show()
```

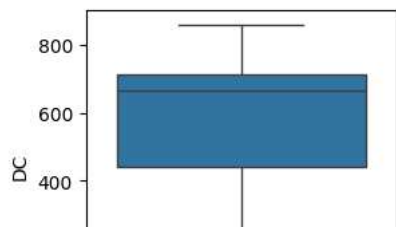
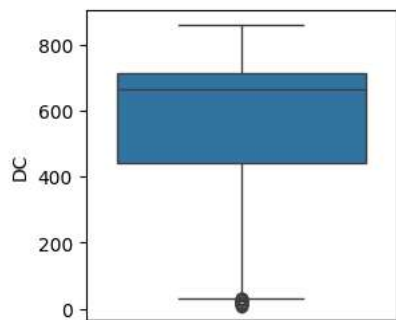

La columna: FPMC tiene 53 outliers
Rango inf: 86.15 Rango sup: 96.95000000000002



La columna: DMC tiene 17 outliers
Rango inf: -42.10000000000002 Rango sup: 253.10000000000002

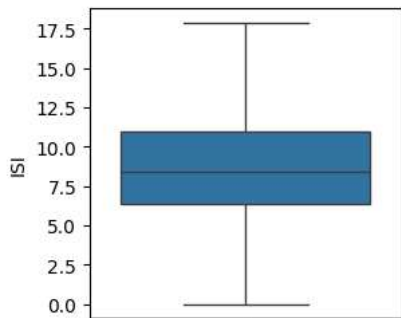
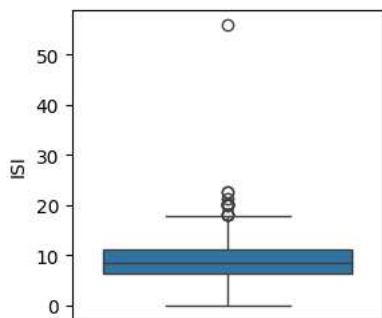


La columna: DC tiene 17 outliers
Rango inf: 31.399999999999977 Rango sup: 1123.4

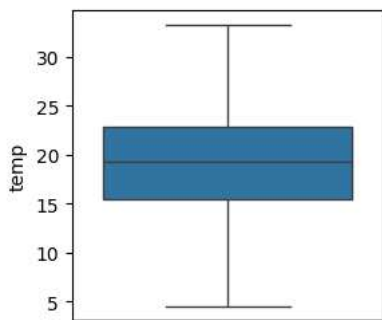
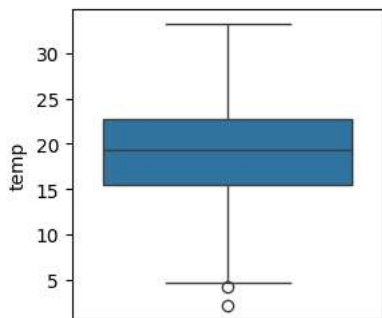




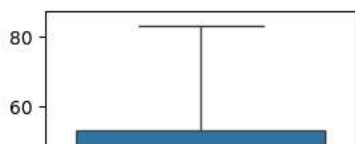
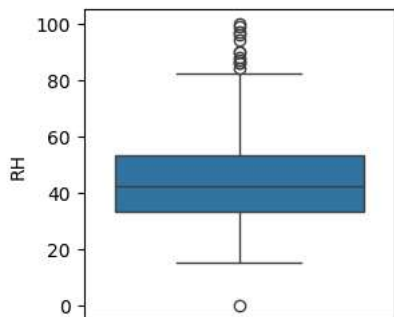
La columna: ISI tiene 14 outliers
Rango inf: -0.4999999999999991 Rango sup: 17.9

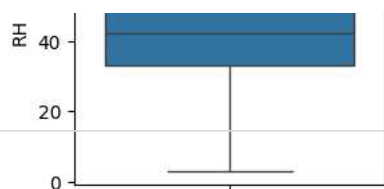


La columna: temp tiene 2 outliers
Rango inf: 4.549999999999999 Rango sup: 33.75

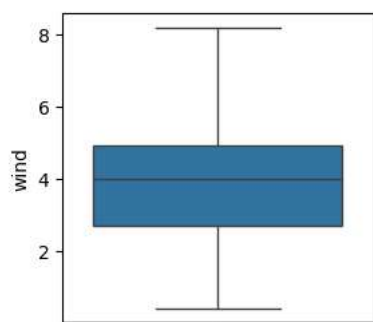
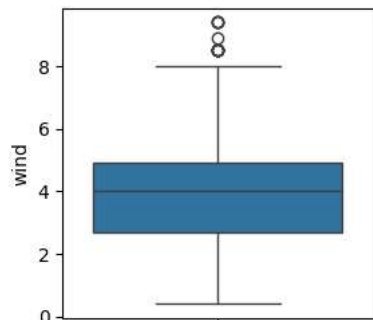


La columna: RH tiene 13 outliers
Rango inf: 3.0 Rango sup: 83.0

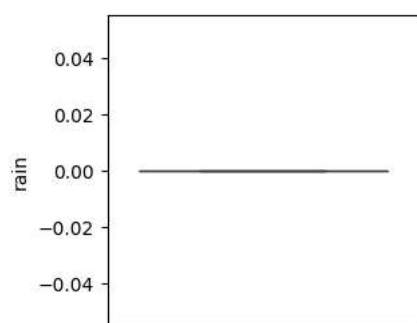
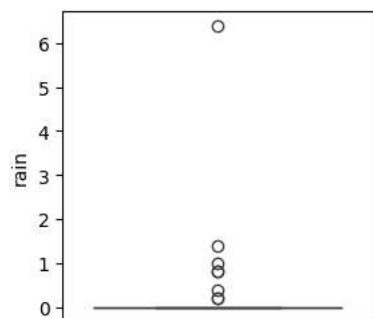




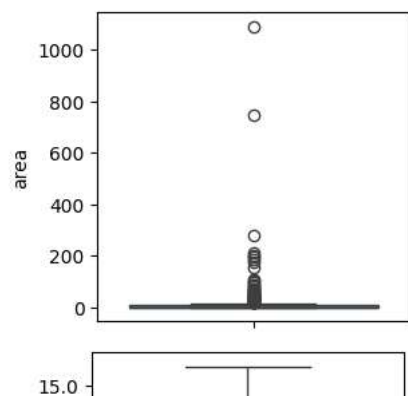
La columna: wind tiene 13 outliers
Rango inf: -0.6000000000000001 Rango sup: 8.200000000000001



La columna: rain tiene 8 outliers
Rango inf: 0.0 Rango sup: 0.0



La columna: area tiene 64 outliers
Rango inf: -9.66 Rango sup: 16.1



```
df.describe()
```

	X	Y	FFMC	DMC	DC	ISI	temp	RH	wind	rain	area
count	513.000000	513.000000	513.000000	513.000000	513.000000	513.000000	513.000000	513.000000	513.000000	513.0	513.000000
mean	4.678363	4.300195	91.300097	110.158480	549.481676	8.896296	18.892788	44.074672	3.995127	0.0	3.959435
std	2.320059	1.230925	2.533121	62.116939	247.029536	3.953742	5.814190	15.835531	1.756327	0.0	5.700034
min	1.000000	2.000000	86.150000	1.100000	31.400000	0.000000	4.550000	3.000000	0.400000	0.0	0.000000
25%	3.000000	4.000000	90.200000	68.600000	440.900000	6.400000	15.500000	33.000000	2.700000	0.0	0.000000
50%	4.000000	4.000000	91.600000	108.300000	664.500000	8.400000	19.300000	42.000000	4.000000	0.0	0.520000
75%	7.000000	5.000000	92.900000	142.400000	713.900000	11.000000	22.800000	53.000000	4.900000	0.0	6.440000
max	9.000000	9.000000	96.200000	253.100000	860.600000	17.900000	33.300000	83.000000	8.200000	0.0	16.100000

```
#RAIN TIENE CASI TODOS CEROS
cantCeros=(df['rain']==0).sum()
print("Cantidad de ceros: ",cantCeros)
print("Procentaje de ceros: ",cantCeros/(len(df)))
```

Cantidad de ceros: 513
Procentaje de ceros: 1.0

```
df=df.drop(columns='rain')
```

```
#AREA TIENE MUCHOS CEROS
cantCeros=(df['area']==0).sum()
print("Cantidad de ceros: ",cantCeros)
print("Procentaje de ceros: ",cantCeros/(len(df)))
```

Cantidad de ceros: 246
Procentaje de ceros: 0.47953216374269003

```
# RESPUESTA
categoricalColumnas=df.select_dtypes(include=['object']).columns
print(f" Variables categóricas encontradas: {list(categoricalColumnas)}")
```

Variables categóricas encontradas: ['month', 'day']

```
df['month'] = np.where(df.month == 'jan', 1, df.month)
df['month'] = np.where(df.month == 'feb', 2, df.month)
df['month'] = np.where(df.month == 'mar', 3, df.month)
df['month'] = np.where(df.month == 'apr', 4, df.month)
df['month'] = np.where(df.month == 'may', 5, df.month)
df['month'] = np.where(df.month == 'jun', 6, df.month)
df['month'] = np.where(df.month == 'jul', 7, df.month)
df['month'] = np.where(df.month == 'aug', 8, df.month)
df['month'] = np.where(df.month == 'sep', 9, df.month)
df['month'] = np.where(df.month == 'oct', 10, df.month)
df['month'] = np.where(df.month == 'nov', 11, df.month)
df['month'] = np.where(df.month == 'dec', 12, df.month)
```

```
df['day'] = np.where(df.day == 'mon', 1, df.day)
df['day'] = np.where(df.day == 'tue', 2, df.day)
df['day'] = np.where(df.day == 'wed', 3, df.day)
df['day'] = np.where(df.day == 'thu', 4, df.day)
df['day'] = np.where(df.day == 'fri', 5, df.day)
df['day'] = np.where(df.day == 'sat', 6, df.day)
df['day'] = np.where(df.day == 'sun', 7, df.day)
```

```
from sklearn.preprocessing import LabelEncoder
def treat_categoricals(df, cols):
    nueva_df=df.copy()
    for c in cols:
        unique_values=nueva_df[c].nunique()
        if np.unique_values<=4:
            nueva_df=pd.get_dummies(nueva_df,columns=[c],prefix=[c])
        else:
            le=LabelEncoder()
            nueva_df=le.fit_transform(nueva_df[c])

    return nueva_df
```